

ARTICLE TYPE

Anomaly Detection on OpenStack Logs Based on an Improved Robust PCA Model and Its Projection onto Column Space

Parisa Sadat Kalaki | Alireza Shameli-Sendi* | Behzad Khalaji Emamzadeh Abbasi

¹Faculty of Computer Science and Engineering, Shahid Beheshti University (SBU), Tehran, Iran

Correspondence

*Alireza Shameli-Sendi. Email: a_shameli@sbu.ac.ir

Summary

With the advent of technology and the development of more complex software systems, the size of logs generated by these systems has increasingly risen so that the anomaly detection for remediating common errors has been more difficult than ever. The cloud emergence in the information technology (IT) industry has led to the immigration of enterprises towards it, which has extended the application of cloud management stacks such as OpenStack. By using the OpenStack platform, users can access resource infrastructure and manage virtual machines (VMs). The anomaly detection in OpenStack logs is not realized conveniently due to the substantial size of logs, and it is required to automate this process. Since there is no appropriate open-source dataset for OpenStack logs, we have generated 25,000 logs by injecting three types of anomalies to propose a more efficient technique in terms of performance and time in detecting anomalies in OpenStack logs relative to recent studies by proper OpenStack log parsing and analyzing these logs by data mining algorithms. To this end, compared to the previous research study, we could improve the anomaly detection performance in terms of F1 score, recall, and precision by 9%, 4%, and 14%, respectively, and decrease the running time relative to the log size by at least 30 seconds.

KEYWORDS:

OpenStack, Log Analysis, RPCA Algorithm, Cloud Computing

1 | INTRODUCTION

Among the complex distributed systems, cloud management stacks are one of the critical elements in cloud computing¹. One of the popular cloud management stacks is OpenStack platform. This platform is an open-source cloud platform composed of different components for VM management in public or private clouds, and different users extensively utilize it due to providing cloud services. It is an infrastructure as a service written by Python programming language. This platform consists of seven core components: *Horizon*, *Nova*, *Glance*, *Neutron*, *Cinder*, *Keystone*, and *Swift*².

OpenStack is prone to performance anomalies that may take hours or days to be detected by system administrators³. When a VM is created in OpenStack software, it must completely go through a life cycle from creation to destruction; otherwise, an anomaly may have occurred. For example, if the VM is destroyed by libvirt¹ on the host server immediately after creating without going through operations such as pause, stop, start, suspend, resume, an anomaly will be recognized in the VM^{4,5,6}.

¹An open-source API library for managing virtualization platforms

Anomaly detection is a process to find unusual patterns which cause abnormal behavior. This process is a critical task for each system. For monitoring a system, logs contain some data about the momentarily status of the system; also, because of the increasing complexity of systems, a huge volume of log data will be generated by them, and this can make the anomaly detection more challenging even for experienced developers, as the conventional techniques such as keyword searching will not work anymore. Therefore, the analysis of system logs is an essential task for understanding the system performance and the cause of its abnormal behavior.

Since the current version of OpenStack logs is not appropriate and the anomaly labels in VMs are unspecified⁷, about 25,000 logs, including normal logs and those with labeled anomalies, are generated in 15 hours and utilized in this study. A total of 1,162 virtual machines have been created, 180 of them have produced anomalies and the rest are normal. This dataset² belongs to Nova which is the most crucial components of OpenStack. If an anomaly related to the VM life cycle occurs in this software, its log data are recorded in Nova; that is why we have considered the log data of this component.

After generating a thousand logs and injecting various anomalies, four steps are performed on the data: 1) log parsing, 2) data preprocessing, 3) log analysis, and 4) anomaly detection. Providing a good analysis of logs requires a structured format for the logs, since most data mining algorithms require structured inputs. Therefore, in the parsing stage, we convert the collected raw logs into structured logs. In log parsing, different algorithms such as *Simple Log File Clustering Tool* (SLCT)⁸, *Abstracting Execution Logs* (AEL)⁹, *Drain*¹⁰, *MoLFI*¹¹, *LogCluster*¹², and *Iterative Partitioning Log Mining* (IPLM) are developed for automating the log parsing operation and obtaining event template, as the manual log parsing is a time-consuming and even infeasible operation due to the enormous volume of log data. IPLM was used in this study for log parsing since it resulted 87% accuracy among other algorithms.

In the log analysis phase, different data mining techniques are utilized for detecting anomalies. In¹³, three supervised techniques - including logistic regression, decision tree, and support vector machine - and three unsupervised techniques - including *Principal Component Analysis* (PCA), log clustering, and invariant mining - are presented and tested for logs of *Blue Gene/L* (BGL) and *Hadoop Distributed File System* (HDFS) systems. It is found that the PCA algorithm has no good performance over BGL datasets. PCA algorithm is sensitive to noise. Since the anomaly plays an important role in detecting anomalies in our data, the *Robusted Principal Component Analysis* (RPCA) method works better in this scenario. In recent years, this algorithm has attracted a lot of attention due to its application in various fields. RPCA is an improved PCA model for noise sensitivity. RPCA is used in closed-circuit television camera images and videos to detect moving objects, eliminate noise, isolate foreground from background, pre-process data, predict protein interactions, and analyze traffic flow data matrices on highways^{14?}.

Another anomaly detection method that has recently been performed on OpenStack logs is the DeepLog method, which has performed better than the PCA algorithm. DeepLog algorithm is one of the methods that LSTM neural network has been used on OpenStack logs and has achieved good results but has high computational complexity¹⁵.

In the present study, logs were generated, and three types of anomalies were injected into datasets. By using the RPCA algorithm, logs were parsed into two low-rank and sparse matrices. Indeed, the low-rank matrix forms the normal space of logs, and the sparse matrix forms the anomalies space. In order to obtain anomalies in the sparse matrix, an appropriate threshold is recognized and used to distinct the anomalies. Since RPCA is an offline algorithm and cannot be used for new data, the projection of the low-rank matrix onto the column space can be applied for every new data to specify its status, and there is no need to repeat the matrix decomposition process for it.

Our contributions in this study are as follows:

- Since there is no suitable dataset of OpenStack logs, 25,000 logs³ accompanied by injection of three anomalies are generated in 15 hours. The 982 normal VMs and 180 VMs with anomalies are created individually.
- RPCA is a batch method and cannot be used on new data, since it has to do the matrix decomposition for the whole dataset. This makes it computationally expensive. In this study, we compute the low-rank matrix obtained from the train dataset, just once at the beginning, then we project every new data point onto the column space of the low-rank matrix. It greatly reduces the computational complexity of this method and gives it an advantage over other methods, such as the DeepLog method, which happens to be one of the best methods for anomaly detection in recent years.

²<https://github.com/ParisaKalaki/OpenStack-logs>

³<https://github.com/ParisaKalaki/OpenStack-logs>

- We improved the anomaly detection in OpenStack logs in terms of F1-score, recall, and precision relative to recent studies applying DeepLog and PCA algorithms. Also, the running time relative to the amount of log data has been reduced by 30 seconds compared to the DeepLog method.

The remainder of this paper is organized as follows. In Section 2, some research studies conducted on anomaly detection in OpenStack logs are reviewed. Section 3 describes the proposed model with details. The proposed model is evaluated and implemented for the generated dataset in Section 4. Finally, the concluded remarks of the evaluation are presented in Section 5.

2 | RELATED WORK

Since this study covers different techniques, the related works are divided into several categories. These categories pointed out in the following include log parsing, log mining techniques for detecting anomalies, and anomaly detection in OpenStack logs.

2.1 | Log Parsing

It is not easy to extract useful information from raw logs since they do not have a structured format. Generally, raw logs must first be converted into structured logs in order to be analyzed by data mining algorithms. In log parsing, there are many algorithms such as IPLOM¹⁶, SLCT¹⁷, Logsig¹⁸, LKE¹⁹, Spell²⁰, Drain¹⁰. Using these algorithms, each event template can be extracted from unstructured logs, and raw logs can be transformed into properly structured logs.²¹ has implemented the log parsing algorithms on log data of different systems such as Spark, Hadoop, HDFS, OpenStack, and others. Every algorithm has different accuracy depending on the dataset used. IPLOM algorithm with 87% accuracy resulted in the highest accuracy among other algorithms implemented on the OpenStack dataset. For this reason, the present study has used IPLOM for log parsing.

IPLOM algorithm follows a four-step partitioning operation. In the first step, we partition logs of the same length into one cluster. This is because logs with the same number of tokens have the same logs format. The previous step gave us a number of clusters with the same number of tokens. In the second step, we select a position from each cluster. If there is a small number of variable tokens in that position, that position probably has constant tokens. So, the logs in that position that have the same constant token, fall into one cluster. During the third stage, logs in a cluster are grouped according to the relationships between their tokens in two positions. The relationships may be one-to-one, one-to-several, multi-to-one, and multi-to-multiple. A heuristic algorithm performs the partitioning depending on the type of relationship. In the last step, clusters of these divisions are obtained, each cluster being obtained by a line format. In any cluster of tokens, if there is only one unique type of token, that token is fixed, and if there is more than one token, that token is variable¹⁶. Therefore, the present study uses the IPLOM algorithm as a tool for parsing raw logs to achieve structured logs, including columns named levelName, component, requestId, content, eventId, eventTemplate, and parameterList specifically for our dataset.

2.2 | Log Mining Methods

There are many methods for detecting anomalies in log data. In¹³, three supervised and three unsupervised techniques were implemented on HDFS and BGL logs. PCA was one of the algorithms used in this paper with high accuracy for HDFS logs, but a low accuracy was obtained for BGL logs. Indeed, this algorithm could not exhibit a good performance considering different values for threshold resulted in the discrimination between anomalies and normal logs.

Lakhina et al.²² also used PCA to detect anomalies in network traffic. An analysis based on principle component analysis is presented for detecting traffic anomalies across networks using a method called subspace method. There is a normal part of the network that is under controlled traffic, and a noisy part that contains a lot of traffic.

In the detection of anomalies, machine learning methods play a crucial role. Unlike the linearly operated PCA, the auto-encoder uses a nonlinear subspace to capture all data, including anomalies²³. Also, CNN and LSTM neural networks are used for OpenStack logs and discussed more in Section 2-3. In this paper, since PCA is susceptible to noises and does not involve abnormal space, RPCA is used as the robust version of this algorithm. On the other hand, RPCA cannot be applied to test data; it is not implemented for inductively detecting anomalies. RPCA is an algorithm that has gained attention recently because of its applications.

One of the main applications of this algorithm is for data preprocessing. In²⁴, RPCA was applied to extract critical features of high dimensional data, and then the SVM algorithm is used for predicting the protein structure leading to a 5% increase in

the accuracy criterion. Also, another important application of this algorithm is to evaluate the closed-circuit television (CCTV) cameras. Recognizing the movement of objects in the camera depends on the distinction between background and foreground, which is conducted by decomposing the associated matrix into two low-rank and sparse matrices²⁵.

In addition to detecting abnormalities within a computer network, RPCA is used to detect cyber attacks. A common problem with the PCA algorithm is that it does not work well in certain situations, such as anomaly detection. As the number of noise data increases, the proper principal components cannot be found because the noise causes the components to deviate from the original direction. Consequently, the PCA method cannot locate the appropriate subspace where normal data is located for this type of dataset, which requires an RPCA algorithm that can withstand noise. In this paper, RPCA is used to analyze closed network data and obtain two low-ranking and privacy matrices. The optimization problem requires the setting of a threshold for a parameter called, which affects individual values in a lower-order matrix²⁶.

²⁷ has applied RPCA to recognize essential events in metro stations and estimate the level of crowdedness in operating metro systems. The DB-scan algorithm and tweet data at that time were used to identify clusters of anomalies based on the sparse matrix S .

RPCA also can be used for preventing confidential information leaks in time-series data. The anomalies can be detected by decomposing a matrix where the rows and columns represent users and the number of user requests in one hour, respectively, and estimating the sparse matrix S ²⁸.

2.3 | Studies conducted on OpenStack

HANSEL² is a monitoring system that examines and analyzes the related network messages and system status to find the root of errors in OpenStack operations. By applying HANSEL on OpenStack and evaluating it, an accuracy of 98% can be achieved. GRETEL is also a system similar to HANSEL, and its difference is that some features are improved in GRETEL. For example, GRETEL recognizes the performance anomalies in addition to operational ones²⁹.

DeepLog⁴ is one of the machine learning methods which is applied on OpenStack logs. This method employs a neural network model named long short-term memory (LSTM) to detect anomalies. It could outperform the PCA, N-gram, and Mining Invariant. Based on the workflow model, this method examines the sequences of events. The model predicts that, if the sequence of events occurs as predicted, then it is normal and otherwise abnormal. DeepLog is learned only based on normal logs and has a high computational complexity.

Recently, Hashemi and Mäntylä⁷ has been applied deep learning for detecting log anomalies. A convolutional neural network (CNN) is a neural network model implemented on HDFS, BGL, and OpenStack datasets. The OpenStack dataset used in this paper is the only set of OpenStack logs in open-source form. As reported in this paper, the intended model has presented poor results for the OpenStack dataset relative to HDFS and BGL; the reason is the lack of a proper dataset of OpenStack logs, and it is hoped that a better version of OpenStack logs to be generated for this field of study.

Yuan et al.³⁰ has investigated the failure diagnosis in the unsuccessful implementation of cloud operating systems on OpenStack logs. This study has only addressed the failure diagnosis for each task focusing on the error logging levels, not on all logs generated for tasks as conducted in this study. Additionally, this work uses a model called word2sec instead of parsing logs. For past failure logs, they created four classification models like K-nearest neighbor, Naive Bayes, Random Forest and Neural Network using natural language processing. Whenever a new failure occurs, the appropriate classification is selected in order to identify the root cause.

In this study, IPOLM is used for parsing OpenStack logs because it has the highest accuracy among all the parsing algorithms that can be implemented on OpenStack logs. As pointed out in one recent study, the current version of the OpenStack dataset produces poor results, as there is no appropriate dataset for parsing and analyzing the OpenStack logs.

Generally, there are also various methods for anomaly detection in logs of different systems, namely DeepLog, that outperforms PCA, which cannot be a suitable algorithm for abnormal space detection because of its sensitivity to noisy data. However, RPCA can eliminate the PCA weakness with matrix decomposition; on the other hand, since this algorithm cannot be implemented on test data, projection onto the column space is utilized to implement it on new data. DeepLog is a method with good results from its implementation on OpenStack logs; in the following, by comparing the proposed method with this algorithm, we will present an enhancement in F1 score and recall and a decrease in the running time relative to the log size.

3 | PROPOSED MODEL

To be able to analyze raw OpenStack logs, we need to parse and structure them first. Additionally, since there is no proper dataset of logs, before performing the analysis, we must create a dataset of OpenStack logs and inject anomalies into it. Once we have generated the logs and injected the anomaly into it, we need to be able to identify the virtual machines in the OpenStack that caused the anomalous behavior. Identifying anomalous patterns in logs requires an ability to separate the normal space from the space in which the anomaly occurs. In order to accomplish this, we use data mining algorithms.

The proposed model includes four main steps that start from raw log parsing followed by different techniques for data preprocessing to prepare them, and finally, the analysis of data and anomaly detection. In the following, we will describe our proposed model in more detail.

3.1 | Log Parsing

The datasets used in this study consist of normal and abnormal logs of Nova component. The details of the dataset are described in section 4.1. Figure 1 shows the general workflow that is addressed in the following. In this step, by applying the IPLOM algorithm, in addition to obtaining a structured log, an event template is also extracted from the logs. There are four steps in IPLOM algorithm. First, it partitions logs based on the number of tokens, so that logs with the same length are placed together in one partition. Logs with the same length are more likely to be similar. In the second step, we partition the data based on token positions. In the logs of a partition obtained from the first step, the position with the least number of variable tokens is most likely to have a constant token, so we put the logs in that position in the same partition. As a third step, every two token positions are partitioned according to one-to-one, one-to-many, many-to-one and multi-to-multiple relationships. Finally, each partition is a cluster. As a result, in each cluster the positions with more than one type of token are variable, and otherwise the tokens are constant. For example, the sentence “Took 69.62 seconds to deallocate network for instance” is transferred into “Took <*> seconds to build instance,” the variables are removed, and other parts are kept intact as event template. More details on this algorithm, can be found in¹⁶. A pattern or event template is obtained for each log, which may be repeated many times for each VMs ID. Two examples of extracting event templates from raw logs are shown in Figure 2.

It is necessary to adjust the IPLOM algorithm according to the type of input logs. Two key factors are identified for each type of logs: (1) The first is the logs structure and (2) the other one is Regex, which varies depending on the logs of each system.

The OpenStack log is structured into eight parts: (1) A timestamp, which shows when the logs were recorded. (2) A log level indicates the severity of a log; the levels are debug, info, warn, error, and fatal. Info level are selected for our dataset. (3) The third part contains the name of an OpenStack component whose logs are displayed. (4) The fourth element is the request ID, for example, the request to create a virtual machine that is sent from the Horizon component to the Nova component. (5) The fifth part is the log message, which includes the virtual machine ID and a description of what happened. (6) The event ID is a unique identifier that is assigned to each of the event templates and is used to construct the feature matrix. (7) The event template, as shown in Figure 2, is extracted from the raw logs. (8) The last element is a list of parameters in the log, for example, the virtual machine ID or time.

Table 1 OpenStack log structure after IPLOM algorithm

TIMESTAMP	2021-04-12 17:41:57.386
LEVEL	INFO
COMPONENT	nova.compute.manager
REQUEST-ID	ac9f5721-5c52-4ec3-ba8a-e494d9780d53
CONTENT	instance: a08d978c-e6df-4fc8-84bd-d510908a08fd VM Started (Lifecycle Event)
EVENT-ID	d3972da2
EVENTTEMPLATE	instance <*> VM Started (Lifecycle Event)
PARAMETERLIST	['a08d978c-e6df-4fc8-84bd-d510908a08fd']

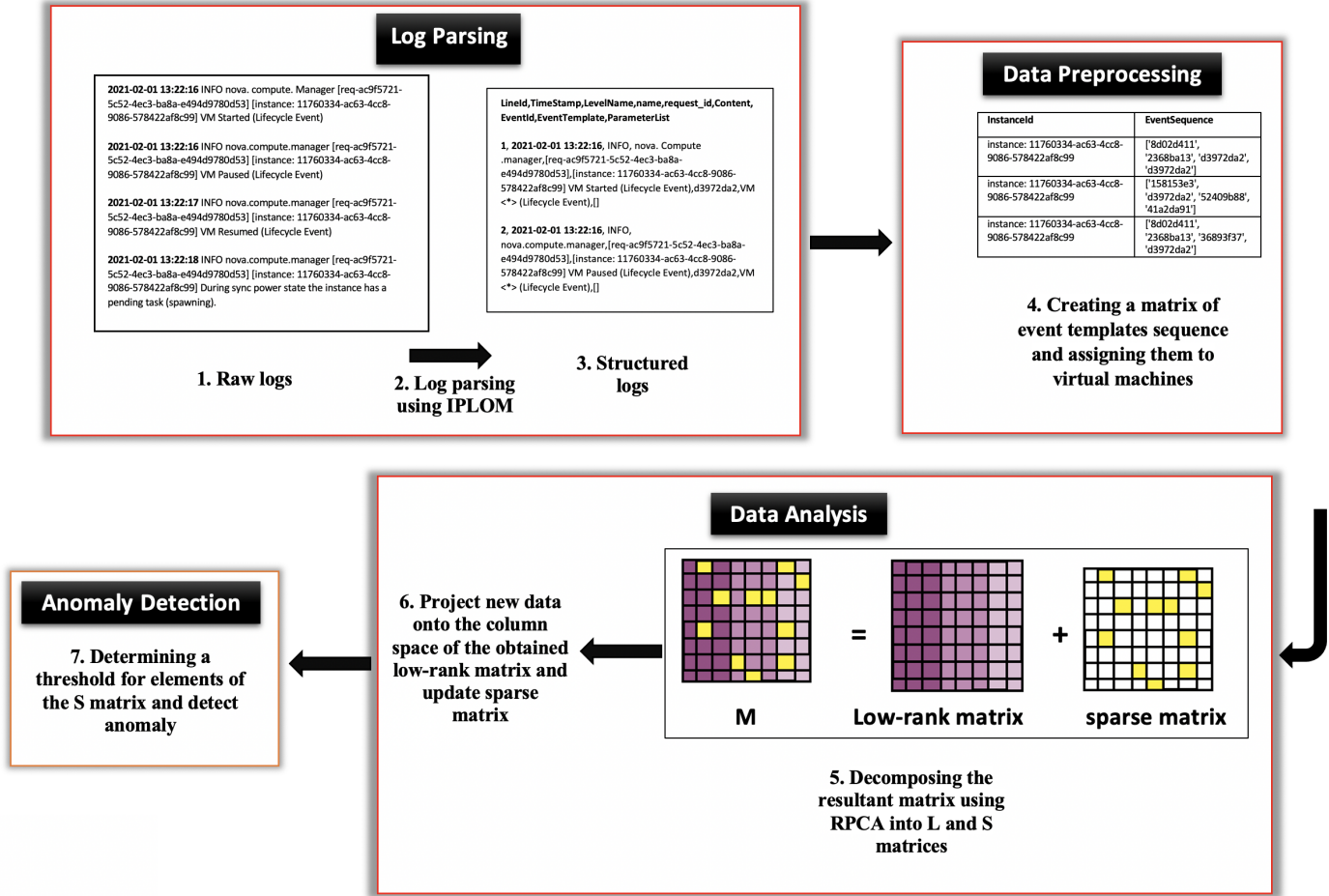


Figure 1 Workflow of the proposed model

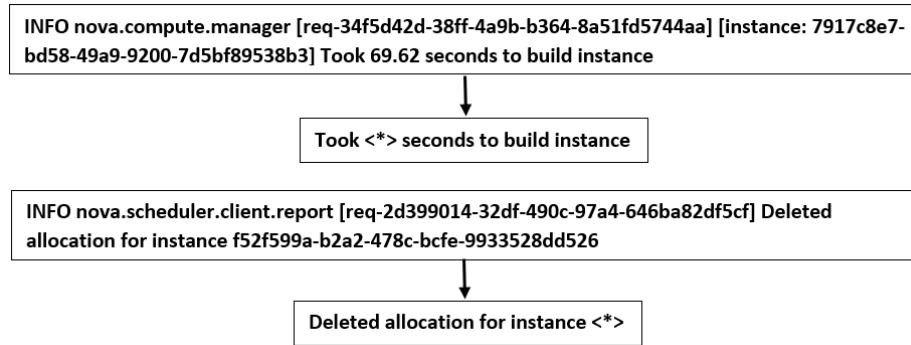


Figure 2 Extracting event templates from logs

3.2 | Data Preprocessing

In order for logs to be automatically analyzed, they must be converted into appropriate formats that can be used as input for data mining algorithms. A number of important and practical features are selected from existing features and also new ones are created. We select the content property, which contains the message content and the IDs of the virtual machines. We do not need the date and time for this method, so we delete it. To start the data analysis, we categorize the logs into windows by identifying the instance id of the virtual machine in OpenStack logs. This is known as the session window. Each unique identifier is thus

Index	InstanceId	EventSequence
505	instance: a96420a7-18fb-4d6e-8d50-ac2487700bea	['8d02d411', '2368ba13', 'd3972da2', 'd3972da2', '158153e3', 'd3972da2', '52409b88', '41a2da91', '158153e3', '7824976e', 'ce0cacf4', '52409b88', 'a79db41b', '4e0f7ce8', '41a2da91', '36893f37', 'd3972da2']
586	instance: 3408b65b-0e57-40fa-9601-e1ccfbd55078	['8d02d411', '2368ba13', 'd3972da2', '52409b88', '41a2da91', '158153e3', 'd3972da2', 'd3972da2', '7824976e', 'ce0cacf4', '52409b88', 'a79db41b', '4e0f7ce8', '41a2da91', '28f25744', '36893f37', 'd3972da2']
642	instance: 3fc8dad6-c1e8-4ee6-bc6d-09791674a13d	['8d02d411', '2368ba13', 'd3972da2', 'd3972da2', '158153e3', 'd3972da2', '52409b88', '41a2da91', '158153e3...
12	instance: e4faecd7-54e4-470b-83af-8c021fb864d7	['8d02d411', '2368ba13', 'd3972da2', '52409b88', '41a2da91', '158153e3', 'd3972da2', '7824976e', 'd3972da2', 'ce0cacf4', '52409b88', 'a79db41b', '4e0f7ce8', '41a2da91', '28f25744', '36893f37', 'd3972da2']
274	instance: 047816c0-642c-46dc-a3b4-25b7831cdad7	['8d02d411', '2368ba13', 'd3972da2', 'd3972da2', '158153e3', 'd3972da2', '52409b88', '41a2da91', '7824976e', 'ce0cacf4', '52409b88', 'a79db41b', '4e0f7ce8', '41a2da91', '28f25744', '36893f37', 'd3972da2']
1019	instance: 5c7d0904-40e7-4602-aa15-013020622d47	['8d02d411', '2368ba13', 'd3972da2', 'd3972da2', 'd3972da2', 'd3972da2', 'd3972da2', 'd3972da2', 'd3972da2...
598	instance: 7f88e2b2-e0d5-4dbf-a647-3fdcf9942cfff	['8d02d411', '2368ba13', 'd3972da2', 'd3972da2', '158153e3', 'd3972da2', '52409b88', '41a2da91', '158153e3...
1116	instance: e3704f12-9c67-4dbc-9c8a-ff413742f3d4	['8d02d411', '2368ba13', 'd3972da2', 'd3972da2', '158153e3', 'a79db41b', '4e0f7ce8', 'b048663e', 'ce0cacf4', '52409b88', '4e0f7ce8', '41a2da91', '54a4aae1', '5d38d5ed', '36893f37', 'd3972da2']
734	instance: 99e44479-566d-41ef-a0ec-ef5b2ad616e2	['8d02d411', '2368ba13', 'd3972da2', '52409b88', '41a2da91', '7824976e', '158153e3', 'd3972da2', 'd3972da2', 'ce0cacf4', '52409b88', 'a79db41b', '4e0f7ce8', '41a2da91', '28f25744', '36893f37', 'd3972da2']
995	instance: 91530882-fea9-4305-a6a3-7340d5a6ea74	['8d02d411', '2368ba13', '43a43c9c', 'd3972da2', 'd3972da2', 'd3972da2', 'd3972da2', 'd3972da2', 'd3972da2...
885	instance: 668998a4-553a-4f86-9c62-fce3a71fc30a	['8d02d411', '2368ba13', 'd3972da2', 'd3972da2', '158153e3', 'd3972da2', '52409b88', '41a2da91', '158153e3', '7824976e', 'ce0cacf4', '52409b88', 'a79db41b', '4e0f7ce8', '41a2da91', '36893f37', 'd3972da2']
630	instance: ade8cdc0-d975-42ed-abe9-a21b7b71c4c6	['8d02d411', '2368ba13', 'd3972da2', '52409b88', '41a2da91', '7824976e', 'd3972da2', 'd3972da2', 'ce0cacf4', '52409b88', 'a79db41b', '4e0f7ce8', '41a2da91', 'd3972da2', '28f25744', '36893f37']

Figure 3 A matrix made up of sessions and event templates

associated with a number of event templates. We categorize all the event templates that have occurred for each ID, and each ID may have a variety of event templates. By the end, a matrix showing rows of sessions, and columns of event templates, is created as shown in Figure 3.

3.2.1 | TF-IDF

Term Frequency-Inverse Document Frequency (TF-IDF) is a statistical approach for weighting the extracted feature vectors that can be used to determine each term's significance in a document. TF is the ratio of the number of times a term occurs in a document to the total number of terms. On the other hand, all terms with high frequency have low significance. Therefore, IDF calculates the logarithm of documents containing the intended term to neutralize the effect of terms with high frequency and low significance.

3.2.2 | zero-score Normalization

A common method of data normalization is the Z-Score method, which relies on standard deviation and mean. In the following formula, x and x' are old and new data, and $\bar{A}$ is the average of the data and σ is the standard deviation. This will allow us to normalize the feature matrix values obtained in the previous step.

$$x' = \frac{x - \bar{A}}{\sigma_A} \quad (1)$$

3.3 | Data Analysis

In the following, we will discuss how to use PCA, RPCA, ALM, and the data projection onto column space algorithms to propose the PRPCA-CS for data analysis in this research. To begin, we will explain in detail the variables, functions, and notation that are used in the proposed algorithm.

3.3.1 | Notations

In this section, some notations and functions are introduced. Matrices M , L , N , and S are $\in \mathbb{R}^{m \times n}$ which has m vectors with n dimensions.

In mathematics, the term norm refers to the magnitude of a vector or matrix and is denoted by $||\cdot||_p$. There are different types of norms, all of which are calculated as follows:

For every vector $x = (x_1, \dots, x_n)^T \in \mathbb{R}^n$:

$$l_p(x) = \begin{cases} \left\{ \sum_{j=1}^n |x_j|^p \right\}^{\frac{1}{p}}, & \text{if } 1 \leq p < \infty \\ \max_{1 \leq j \leq n} |x_j|, & \text{otherwise } p = \infty \end{cases} \quad (2)$$

For matrix M , frobenius norm ($\|\cdot\|_F$), nuclear norm ($\|\cdot\|_*$) and l_1 norm ($\|\cdot\|_1$) are defined as follows:

$$\|M\|_F = \left(\sum_{i=1}^m \sum_{j=1}^n |m_{ij}|^2 \right)^{\frac{1}{2}} \quad (3)$$

$$\|M\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |m_{ij}| \quad (4)$$

$$\|M\|_* = \sum_i \sigma_i(M) \quad (5)$$

Consider M as a matrix, where $rank(M)$ denotes the number of independent linear columns in the matrix, or the space spanned by its columns. So A matrix rank is determined by the dimension of column space or row space. $rank(M) = r$ implies that r columns in matrix M are independent. Low rank matrix is a matrix with very few independent columns compared to the total column numbers.

Suppose the matrix M is a linear transformation which $Mx = \lambda x$. The vector x is eigenvector and λ is the eigenvalue. So, σ in (5) is a singular value and equal to $\sigma_i = \sqrt{\lambda_i}$, where $(i \leq r)$ ³¹.

The constrained optimization problem is:

$$\begin{aligned} & \min f(X), \\ & \text{subject to } h(X) = 0 \end{aligned} \quad (6)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ can be solved by augmented Lagrange function which is:

$$L(X, Y, \mu) = f(X) + \langle Y, h(x) \rangle + \frac{\mu}{2} \|h(X)\|_F^2 \quad (7)$$

where μ is a positive scalar³².

For matrix recovery, a soft-thresholding operator is applied element-wise on matrices:

$$S_\tau[x] = \begin{cases} x - \tau, & \text{if } x > \tau \\ x + \tau, & \text{if } x < -\tau \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

where $x \in \mathbb{R}$ and $\tau > 0$.

Singular Value Decomposition (SVD) decomposes a matrix into three matrices. This decomposition has many applications in data mining and provides significant insights about linear transformations. Given a matrix M , the SVD is equal to:

$$M = U \Sigma V^T \quad (9)$$

where U is an orthogonal $m \times n$ matrix, V is an orthogonal $n \times n$ matrix, and Σ is a $n \times n$ diagonal matrix which first r row having singular values.

3.3.2 | Recovering Low Rank and Sparse Matrices

PCA is one of the conventional methods for reducing the dimension of data. With the PCA method, we find the direction where the data projection has the most variance or the smallest error rate. To calculate PCA, if we have the matrix M , then:

$$M = L_0 + N_0 \quad (10)$$

L_0 is a low rank and N_0 is a small matrix of noise. PCA method seeks to estimate L_0 with a k rank and find a best low rank matrix. The optimization problem is:

$$\begin{aligned} & \min \quad \|M - L_0\|_F^2 \\ & \text{subject to } rank(L) \leq k \end{aligned} \quad (11)$$

Due to the presence of the Frobenius norm at this point, if noisy data are present, the PCA components depart from their original axes. This makes the PCA algorithm too sensitive to noise. In the case of small noise, the SVD decomposition will

provide the answer, otherwise, we have to try another method. Because of the prominent role played by anomalies in our problem, the RPCA method performs better for detecting anomalies in the data. RPCA is used in CCTV images and videos to detect moving objects, eliminate noise and isolate the foreground from the background³³. In recent years, this algorithm has attracted much attention due to its application in various fields. RPCA is the so-called robust PCA algorithm used to decompose a matrix into two low-rank matrices L and a sparse matrix S that has a number of non-zeros elements:

$$M = L + S \quad (12)$$

For the matrix decomposition of M , we have to solve the minimization problem of (13).

$$\begin{aligned} & \min_{L, S} \|L\|_0, \\ & \text{subject to } M = L + S, \\ & \text{rank}(L) \leq k \end{aligned} \quad (13)$$

k is the best rank to which the dimensions of a matrix can be reduced. There is no efficient solution to this non-convex optimization problem. It has no closed form solution and is a NP-hard optimization problem. Due to the fact that it has been proven that the nuclear norm (sum of singular values) completely recovers the component L and the l_1 -norm recovers the matrix S , so we change the problem of minimization (13) to problem (14). In order to solve the matrix decomposition problem, we need to solve the problem of nuclear norm minimization, which is called the Principal Component Pursuit:

$$\begin{aligned} & \min_{L, S} \|L\|_* + \lambda \|S\|_1, \\ & \text{subject to } M = L + S \end{aligned} \quad (14)$$

where λ is a parameter for balancing the two matrices S and L . $\|L\|_*$ is called nuclear norm of L and equal to sum of singular values. $\|S\|_1$ denotes the ℓ_1 -norm of S . One of these algorithms is the Augmented Lagrange Multiplier Algorithm (ALM), which has a high accuracy compared to the others³³. We can solve (14) by ALM as follows:

$$\begin{aligned} l(L, S, Y) = & \|L\|_* + \lambda \|S\|_1 + \langle Y, M - L - S \rangle + \\ & \frac{\mu}{2} \|M - L - S\|_F^2 \end{aligned} \quad (15)$$

By repeating calculation $(L_k, S_k) = \arg \min_{L, S} l(L, S, Y_k)$ and updating the Lagrange multiplier matrix through $Y_{k+1} = Y_k + \mu(M - L_k - S_k)$, the augmented Lagrange multiplier method can solve the PCP problem (The calculation is summarized in Algorithm 1). In the following, we will calculate $\min_L l(L, S, Y)$ and $\min_S l(L, S, Y)$.

Let $S_\tau : \mathbb{R} \rightarrow \mathbb{R}$ and S_τ be a shrinkage operator for sparsifying the matrix S with the following value:

$$S_\tau[x] = \text{sgn}(x) \max(|x| - \tau, 0) \quad (16)$$

For matrix X , we define the function $D_r(X) = U S_\tau(\Sigma) V^T$ that performs the SVD decomposition on matrix X (Line 2 in Algorithm 1):

$$\arg \min_L l(L, S, Y) = D_{\lambda\mu}(M - S + \mu^{-1}Y) \quad (17)$$

Similarly, this function is applied to all elements of matrix S . Therefore, we have (Line 3 in Algorithm 1):

$$\arg \min_S l(L, S, Y) = S_{\lambda\mu}(M - L + \mu^{-1}Y) \quad (18)$$

Accordingly, the practical approach is to first minimize the l function relative to the L function (with constant S), then minimize the l function relative to S (with constant L), and finally update the Lagrange coefficient Y matrix using the remaining $M - L - S$ (Line 4 in Algorithm 1).

The value of μ in this algorithm is equal to $\mu = n_1 n_2 / 4 \|M\|_1$. The algorithm is continued until $\|M - L - S\|_F \leq \delta \|M\|_F$ and $\delta = 10^{-7}$ ³⁴.

As a result of applying the RPCA algorithm to the OpenStack log set, two matrices of low rank and sparse are obtained. Low-rank matrices constitute normal space, while privacy matrices contain anomalies.

Algorithm 1 Solving the PCP problem using an Augmented Lagrange multiplier

Input: $S_0 = Y_0 = 0, \mu > 0$
Output: S, L

- 1: **while** not converged **do**
- 2: compute $L_{k+1} = D_u(M - S_k - \mu^{-1}Y_k)$
- 3: compute $S_{k+1} = S_{\lambda\mu}(M - L_{k+1} + \mu^{-1}Y_k)$
- 4: compute $Y_{k+1} = Y_k + \mu(M - L_{k+1} - S_{k+1})$
- 5: **end while**
- 6: return S, L

3.3.3 | New Data Projection onto the column space

The RPCA algorithm uses a batch method in which, with each newly-added vector of data, the matrix decomposition is recalculated for the entire dataset, and thus it is not possible to predict the test data. This is because no parameters are obtained from the data training section that can be used immediately for testing.

We therefore project the new data onto the low-rank space we computed earlier, so that we would not have to recalculate the matrix every time the new data is processed. The following is a detailed description.

If M is the data matrix and $M \in \mathbb{R}^{m \times n}$, we assume $M_t \in \mathbb{R}^n$, is the new data vector. Now we apply RPCA to matrix M to create two matrices L and S (See algorithm 2). SVD decomposes matrix L according to the following algorithm and yields the U_1 and U_2 matrices (Line 1 in Algorithm 2).

$$M = U\Sigma V^T = [U_1 U_2] \begin{bmatrix} \Sigma_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1^T \\ V_2^T \end{bmatrix} = U_1 \Sigma_1 V_1^T \quad (19)$$

which $U_1 \in \mathbb{R}^{m \times r}$, $U_2 \in \mathbb{R}^{m \times n-r}$, $V_1 \in \mathbb{R}^{n \times r}$, and $V_2 \in \mathbb{R}^{n \times n-r}$. $u_1, \dots, u_r$ are the bases for the columns space, $u_{r+1}, \dots, u_n$ are the bases for the left null space, $v_1, \dots, v_r$ are the bases for the row space, and $v_{r+1}, \dots, v_n$ are the bases for the null space of the matrix M . As we are looking for a low-rank matrix to separate normal data from abnormal, the first r column of the U -matrix, which forms the column space of the normal space, plays an important role in creating the low-rank matrix. As a result, all the data is projected in column space to reduce its dimension.

For the projection on the matrix U , we know that for every matrix $W \in \mathbb{R}^{m \times n}$, which is a semi orthogonal ($W^T W = I$), its projection on W is equal to (Line 2 in Algorithm 2):

$$P_W = W(W^T W)^{-1} W^T = W W^T \quad (20)$$

As mentioned earlier, we apply RPCA to matrix M to generate two matrices L and S . For this purpose, after obtaining the desired projection on the U matrix, L_{new} is obtained through the projection on M_t vector as follows (Line 3 in Algorithm 2):

$$L_{new} = \text{Projection}_M \cdot M_t \quad (21)$$

Finally, after obtaining L_{new} , S_{new} is calculated as follows (Line 4 in Algorithm 2):

$$S_{new} = M_t - L_{new} \quad (22)$$

3.4 | Anomaly Detection

After determining the sparse matrix S , it is required to determine the threshold for anomaly detection. For each row of S , an element with the maximum value is considered; if this element is higher than the threshold $\forall S_i, \max |S_{ij}| > \alpha$, it takes the anomaly label, otherwise the normal one.

Algorithm 2 Projection Robusted Principal Component Analysis onto the Column Space (PRPCA-CS)**Input:** M_t, L **Output:** S_{new}, L_{new}

- 1: $U, \Sigma, V = \text{SVD}(L)$
- 2: $\text{Projection}_M = U_r \cdot U_r^T$
- 3: $L_{new} = \text{Projection}_M \cdot M_t$
- 4: $S_{new} = M_t - L_{new}$
- 5: return S, L

Table 2 Summary of dataset

#VMs	#Abnormal VMs	#Normal VMs	Time
1,162	180	982	15 hours

4 | EVALUATION

The purpose of this section is to describe the dataset used, the injection of different anomalies, and the evaluation of the proposed model based on different criteria.

4.1 | Dataset

A dataset of OpenStack logs was required to conduct this study. There is only one open-source dataset of OpenStack, which is not useful and reliable due to several issues in logs containing anomalies. Therefore, about 25,000 OpenStack logs are generated along with the fault injection. Each log has an Instance-Id, which shows the VM ID. This ID is unique, and as demonstrated in Table 2, the total number of unique IDs for VMs are separately presented according to being normal, containing anomalies, and also the time required for generating these logs. Due to resource limitations, we were not able to have 1,162 virtual machines at the same time. On the other hand, according to the anomalies considered in the paper, the simultaneous execution of all virtual machines was not necessary. Therefore, after creating a limited set of virtual machines (between 8 to 10 VMs), we delete one or two of them randomly after a period of time, and then a virtual machine is added to the previous set. This routine continues for up to 15 hours to form our dataset.

4.2 | Test Environment

All tests are executed by the Linux server operating system with four intel core i7-4700mq processors and 12 GB Ram on Linux distribution of Ubuntu 18.04.5 LTS and Linux core 4.15.0-143-generic.

4.3 | Fault Injection

The VM life cycle consists of two general sections named configuration and deployment. The configuration section is to create, modify, and test the VM and the deployment section relates to managing, destroying, and monitoring the VM.

The anomaly may occur in each step mentioned above. Three anomalies are generated referring to⁴. For generating logs, the open-source tool *virsh* is utilized to destroy VMs. Virsh sends the VM destruction command to KVM by connecting to libvirt. KVM, as a hypervisor, is responsible for managing the resources and VM life cycle.

Nova is an OpenStack computing component that has responsibility for managing virtual machines. When a virtual machine boot request is issued from the Horizon dashboard, the Keystone component authenticates this request. A request to fetch the image is sent by Nova to Glance if the component is authenticated. As soon as the Nova component receives the image, it requests the Neutron component allocate ports and network. Nova completes virtual machine setup after receiving a response from the neutron component. As mentioned, the Nova component performs coordination between the other components, plays

Table 3 Confusion matrix of applying RPCA-CS on test data

	Predict: Normal	Predict: Abnormal	
Actual: Normal	190 82%	7 3%	197
Actual: Abnormal	3 1%	31 13%	34
	193	38	231

an important role in each stage and is the core of the components. Thus, the logs investigated in our study are related to the Nova component logs.

In the following, three types of injected anomalies are explained:

- VM is destroyed ungracefully right after creation and before completely go through its life cycle.
- After the creation of the VM, its virtual disk is removed from the host server. Unlike the former anomaly where the VM is destroyed, the VM configuration remains unchanged, though it does not have access to the storage space required for booting the operating system.
- A disturbance is applied to the performance of Neutron, which is responsible for managing the network. In this way, decreasing the responsiveness time of this component led to the timeout error, and also by stopping the DHCP service that is charged for assigning IP to VM, the VM network is disturbed practically.

4.4 | Evaluation Criteria

The algorithm is randomly executed on training and test datasets for ten times to evaluate the efficiency of the method described above for the intended dataset, and the average of results is obtained. In our study, we evaluated the performance of anomaly detection algorithms using the F1-Score, Recall, Precision and Accuracy criteria, which are the most frequently used metrics. Prior to that, the concept of the Confusion Matrix is discussed in our study, which is very useful for evaluating models. Matrix confusion is a method for comparing the performance of two classification algorithms.

When there is more data in one class than in another, or if there are more than two classes in the dataset, accuracy alone can lead to misleading results. This study's dataset has more normal class data than abnormal class data. This led us to use the confusion matrix in conjunction with other criteria. The confusion matrix can provide better insight into how well the classification model is working and what errors are being made. The confusion matrix can be calculated based on the number of each of the four types of predictions. There are four elements in this matrix, including false positives, false negatives, true positives, and true negatives. These elements are categorized based on the actual and predicted labels.

Table 3 shows all true positives, true negatives, false positives, and false negatives for OpenStack logs after applying the RPCA-CS model to test data. We can see that the number of normal logs that have been correctly identified as normal has the highest value. In our research, a positive label indicates abnormality, while a negative label indicates normality.

The recall, precision, accuracy, and F1-score criteria formulated in the following are among the critical criteria for measuring the anomaly detection techniques, and they operate based on predicted and real labels. The recall is used to present the ratio of the anomalies detected correctly to the total number of anomalies, precision is applied to indicate the correctness of the detected anomalies, and the F1-score is a combination of the two former criteria. Accuracy is also utilized for measuring the validity of all predictions.

$$\text{Precision} = \frac{\text{number of correctly detected anomalies}}{\text{total number of detected anomalies}}$$

$$\text{Recall} = \frac{\text{number of correctly detected anomalies}}{\text{total number of anomalies}}$$

$$\text{F1-score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Accuracy} = \frac{\text{number of correctly detected data}}{\text{total number of data}}$$

4.5 | Comparing precision and recall criteria

Our approach was to perform the algorithm ten times, each time on a random training and testing dataset, and then average the results. As shown in Figure 4, the three criteria are compared for PCA¹³, DeepLog⁴, and PRPCA-CS. DeepLog is one of the anomaly detection methods that has an acceptable performance on OpenStack logs. To parse logs, the spell parser is used. For modeling, LSTM neural networks have also been used to predict future behavior by analyzing the behavior of past logs.

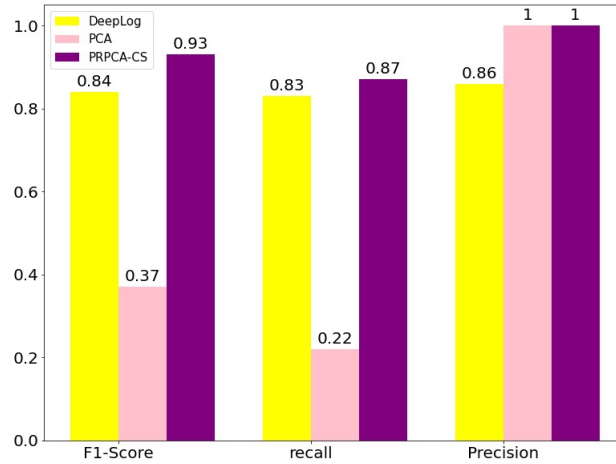


Figure 4 Evaluation of precision, recall, and F1 score criteria on OpenStack logs for PCA¹³, DeepLog⁴, and PRPCA-CS.

In order to get a complete sequence, the model predicts the next step in the sequence of events. The actual event sequence is then compared with the predicted event sequence. When the sequence of events is consistent with predictions, it is normal. If not, it is abnormal. As shown in Figure 4, DeepLog also has a high computational complexity and only uses normal logs to train the model, which is a weakness, whereas our proposed method uses two types of normal and abnormal logs¹⁵. According to the F1 rating criteria, our proposed model performed better than DeepLog in terms of recall and precision.

Among the most popular dimensional reduction algorithms, PCA identifies the principal components and projects data onto them. The principal component is an axis in which the data has the greatest variance. Therefore, one of the drawbacks of PCA is the sensitivity to noise data. Compared to the other two algorithms, the PCA algorithm performed very poorly in terms of recall and F1 score, with 37% and 53%, respectively. Although it performed well in terms of precision, its recall cost is only 37%, since it correctly detected many anomalies, but many normal logs were misdiagnosed as abnormal. As a result of anomalies within logs, the principal components have deviated from their original axes.

PCA seeks to minimize the error rate in Equation $E = \sum_{i=1}^N ||x^n - \bar{x}^n||^2$ represents the original data and $\bar{x}^n$ is its projection on the new coordinates, so for noise data, the presence of 2-norm $||.||^2$ causes the difference between the original data and its projected value to be excessive, and consequently, this difference leads to deviation of the principal component.

4.6 | The accuracy criteria, and running time against log size

As we increase the volume of logs and compare the runtimes of these three algorithms, DeepLog, requires more runtime, however, PCA, and PRPCA-CS have a very sloping linear slope and are relatively close together as shown in Figure 5. We can see that the execution times of PRPCA-CS and DeepLog are at least 30 seconds apart. DeepLog has a high computational complexity that increases execution time, but in the proposed model, using projection data instead of recalculating the matrix decomposing process has led to a significant reduction in computational complexity, which shows a differential in runtime from the DeepLog model.

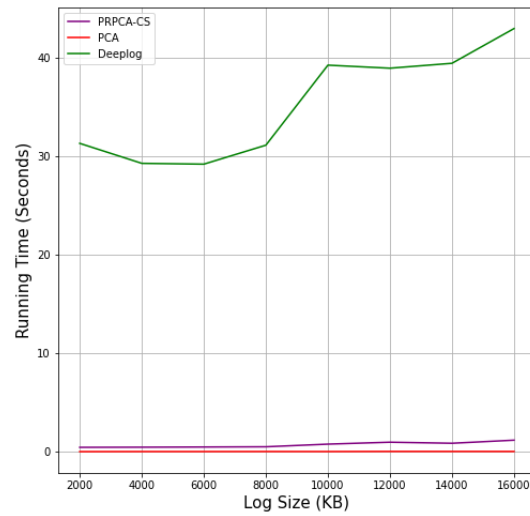


Figure 5 Running time against the log size.

The accuracy criteria of PRPCA-CS, PCA, and DeepLog are shown in Figure 6 to demonstrate that as the volume of logs increases, PRPCA-CS still maintains high accuracy, even in areas where the volume of logs is low, among other algorithms. The DeepLog have very low accuracy at low data volumes compared to the other two algorithms, which could be because there is not enough data for neural network training. The PCA is also slightly better than PRPCA-CS, with a slight difference. This is due to the RPCA decomposition that is performed once at the beginning of PRPCA-CS.

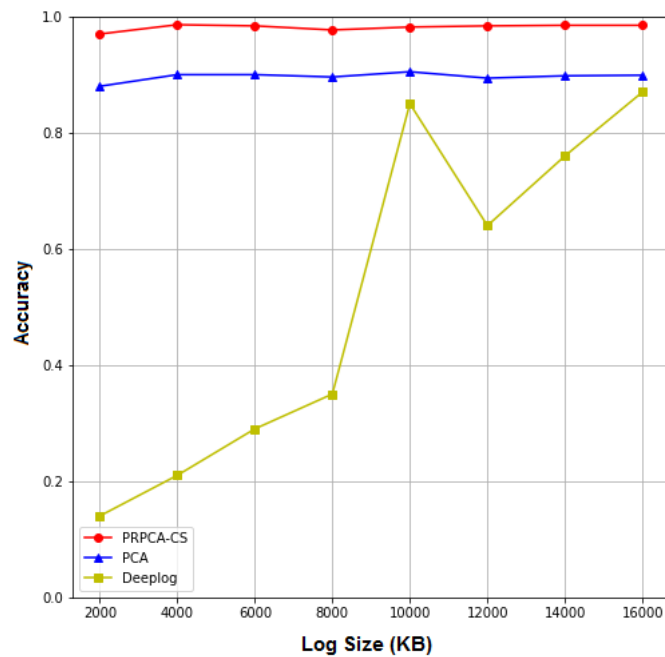


Figure 6 Accuracy of three algorithms versus the log size.

4.7 | Evaluating true positive and false positive rates

In order to compare the rates of correctly diagnosed anomalies with the rates of misdiagnosed anomalies, we use a Receiver Operating Characteristic (ROC) curve diagram. To do this, we calculate $TPR = \frac{TP}{TP+FN}$ and $FPR = \frac{FP}{FP+TN}$ rates.

When TPR has a low value, it means there are a lot of normal data that have been misdiagnosed as abnormal, while FPR refers to the number of normal data that have been correctly detected. In Figure 7, which shows ROC diagrams of the performance of the three algorithms, the y-axis represents the true-positive rate and the x-axis represents the false-positive rate.

The closer the graph is to the $y = 1$ axis, the more favorable it is for us because we have a higher ratio of true positives than false positives, which means that the rate of correct detection of anomalies is higher than the rate of incorrect detection of anomalies. A dotted line indicates a random line, meaning that the results were chosen by chance with a 50% probability, and if the chart is below this line, it is expected to perform poorly. As the area under the curve gets closer to 1, it works better. Figure 7 shows that proposed model has a larger area and better performance, which indicates that it is better able to distinguish between positive and negative classes. The areas of the three algorithms PCA, DeepLog and proposed model are equal to 60%, 87% and 94%, respectively, and the PCA diagram has the lowest area.

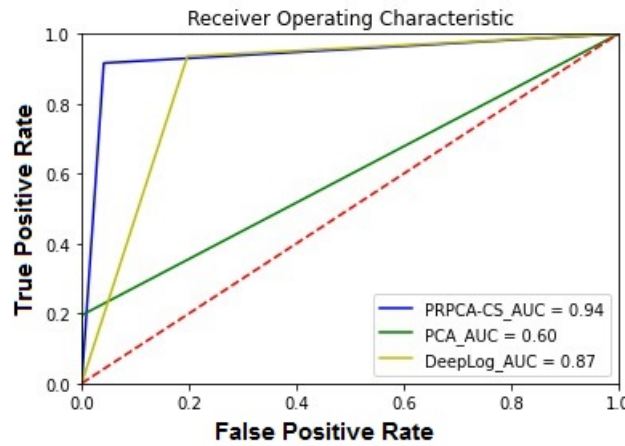


Figure 7 ROC diagram of three algorithms

5 | CONCLUSION

Since cloud computing has contributed to several evolutions in the information technology industry, platforms such as OpenStack have been popular among users for managing virtual machines. However, such platforms are also prone to anomalies, and detecting these anomalies is usually difficult for system administrators because of the huge size of log data or unrecorded logs on the debugging level. Since there is no proper dataset of OpenStack logs, this study generates the required logs and conducts the injection of three types of anomalies. RPCA, as the robust version of PCA, is used to decompose the matrix composed of virtual machines and their features into two low-rank and sparse matrices. For recognizing anomalies in each new data, it is projected onto the column space of the low-rank matrix, and finally, the existing anomalies in the OpenStack log dataset are specified concerning the determined sparse matrix and the proper threshold. The recall and precision criteria are improved compared to the DeepLog algorithm, which is one of the best algorithms implemented on OpenStack logs. Additionally, the execution time has been significantly reduced compared to DeepLog, and the difference is at least 30 seconds in different volumes, and in terms of recall and precision, it has improved by respectively 4% and 14% compared to DeepLog. It is expected that we can test the proposed model on a dataset with a wide variety of anomalies and even logs from other systems.

References

1. Ju X, Soares L, Shin KG, Ryu KD, Da Silva D. On fault resilience of OpenStack. In: ; 2013: 1–16.
2. Sharma D, Poddar R, Mahajan K, Dhawan M, Mann V. Hansel: Diagnosing faults in openStack. In: ; 2015: 1–13.
3. Yu X, Joshi P, Xu J, Jin G, Zhang H, Jiang G. Cloudseer: Workflow monitoring of cloud infrastructures via interleaved logs. *ACM SIGARCH Computer Architecture News* 2016; 44(2): 489–502.
4. Du M, Li F, Zheng G, Srikumar V. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: ; 2017: 1285–1298.
5. Cotroneo D, De Simone L, Liguori P, Natella R, Bidokhti N. How bad can a bug get? an empirical analysis of software failures in the openstack cloud computing platform. In: ; 2019: 200–211.
6. OpenStack issue tracker. <https://bugs.launchpad.net/openstack>; . Accessed: 2022-08-17.
7. Hashemi S, Mäntylä M. OneLog: Towards End-to-End Training in Software Log Anomaly Detection. *arXiv preprint arXiv:2104.07324* 2021.
8. Vaarandi R. Mining event logs with slct and loghound. In: IEEE. ; 2008: 1071–1074.
9. Jiang ZM, Hassan AE, Flora P, Hamann G. Abstracting execution logs to execution events for enterprise applications (short paper). In: IEEE. ; 2008: 181–186.
10. He P, Zhu J, Zheng Z, Lyu MR. Drain: An online log parsing approach with fixed depth tree. In: IEEE. ; 2017: 33–40.
11. Messaoudi S, Panichella A, Bianculli D, Briand L, Sasnauskas R. A search-based approach for accurate identification of log message formats. In: ; 2018: 167–177.
12. Vaarandi R, Pihelgas M. Logcluster-a data clustering and pattern mining algorithm for event logs. In: IEEE. ; 2015: 1–7.
13. He S, Zhu J, He P, Lyu MR. Experience report: System log analysis for anomaly detection. In: IEEE. ; 2016: 207–218.
14. Xing X, Zhou X, Hong H, Huang W, Bian K, Xie K. Traffic flow decomposition and prediction based on robust principal component analysis. In: IEEE. ; 2015: 2219–2224.
15. Farzad A, Gulliver TA. Unsupervised log message anomaly detection. *ICT Express* 2020; 6(3): 229–237.
16. Makanju AA, Zincir-Heywood AN, Milios EE. Clustering event logs using iterative partitioning. In: ; 2009: 1255–1264.
17. Vaarandi R. A data clustering algorithm for mining patterns from event logs. In: Ieee. ; 2003: 119–126.
18. Tang L, Li T, Perng CS. LogSig: Generating system events from raw textual logs. In: ; 2011: 785–794.
19. Fu Q, Lou JG, Wang Y, Li J. Execution anomaly detection in distributed systems through unstructured log analysis. In: IEEE. ; 2009: 149–158.
20. Du M, Li F. Spell: Streaming parsing of system event logs. In: IEEE. ; 2016: 859–864.
21. Zhu J, He S, Liu J, et al. Tools and benchmarks for automated log parsing. In: IEEE. ; 2019: 121–130.
22. Lakhina A, Crovella M, Diot C. Diagnosing network-wide traffic anomalies. *ACM SIGCOMM computer communication review* 2004; 34(4): 219–230.
23. Chalapathy R, Menon AK, Chawla S. Robust, deep and inductive anomaly detection. In: Springer. ; 2017: 36–51.
24. Zakaria NA, Shah ZA, Kasim S. Protein Structure Prediction Using Robust Principal Component Analysis and Support Vector Machine. *International Journal of Data Science* 2020; 1(1): 14–17.

25. Bouwmans T, Zahzah EH. Robust PCA via principal component pursuit: A review for a comparative evaluation in video surveillance. *Computer Vision and Image Understanding* 2014; 122: 22–34.
26. Paffenroth R, Kay K, Servi L. Robust pca for anomaly detection in cyber networks. *arXiv preprint arXiv:1801.01571* 2018.
27. Wang X, Zhang Y, Liu H, Wang Y, Wang L, Yin B. An improved robust principal component analysis model for anomalies detection of subway passenger flow. *Journal of advanced transportation* 2018; 2018.
28. Jin Y, Qiu C, Sun L, Peng X, Zhou J. Anomaly detection in time series via robust PCA. In: IEEE. ; 2017: 352–355.
29. Goel A, Kalra S, Dhawan M. Gretel: Lightweight fault localization for openstack. In: ; 2016: 413–426.
30. Yuan Y, Shi W, Liang B, Qin B. An approach to cloud execution failure diagnosis based on exception logs in OpenStack. In: IEEE. ; 2019: 124–131.
31. Hoffman K, Kunze R, Finsterbusch HE. *Álgebra lineal*. Prentice-Hall Hispanoamericana . 1973.
32. Lin Z, Chen M, Ma Y. The augmented lagrange multiplier method for exact recovery of corrupted low-rank matrices. *arXiv preprint arXiv:1009.5055* 2010.
33. Candès EJ, Li X, Ma Y, Wright J. Robust principal component analysis?. *Journal of the ACM (JACM)* 2011; 58(3): 1–37.
34. Yin W, Osher S, Goldfarb D, Darbon J. Bregman iterative algorithms for l1-minimization with applications to compressed sensing. *SIAM Journal on Imaging sciences* 2008; 1(1): 143–168.

AUTHOR BIOGRAPHY



Parisa Sadat Kalaki. received her B.Sc. and M.Sc. degrees in software engineering from Alzahra University and Shahid Beheshti University (SBU), Tehran, Iran, respectively. During the last five years, she has been actively working in artificial intelligence and data mining fields. During her master, Parisa worked on online anomaly detection on OpenStack logs project. Her research interests lie in the area of cloud computing, anomaly detection, and mathematical optimization.



SBU, in 2018.

Alireza Shameli-Sendi. is currently an Assistant Professor at Shahid Beheshti University. Before joining SBU, he was a Postdoctoral Fellow at Ericsson, Canada and Postdoctoral at ETS and McGill universities in collaboration with Ericsson. He received his Ph.D degree in computer engineering from Montreal University (Ecole Polytechnique de Montreal), Canada. He obtained his B.Sc. and M.Sc. from Amirkabir University of Technology. His primary research interests include information security, intrusion response system, and cloud computing. He is a recipient of Postdoctoral Research Fellowship Award and Industrial Postdoctoral Fellowship Award from Canada. In addition, he received the best researcher award, in industrial track, at



Behzad Khalaji Emamzadeh Abbasi. is currently a Ph.D. student at Shahid Beheshti University, Tehran, Iran. He received his M.Sc. degree in computer engineering from the Isfahan University of Technology, Isfahan, Iran. He obtained his B.Sc. from Bu-Ali Sina University, Hamedan, Iran. His doctoral project is in the field of software code vulnerabilities detection. He has extensive experience in working with machine learning techniques. His interests are in the areas of vulnerability detection, deep learning, and cloud computing.

How to cite this article: Kalaki P. S., A. Shameli-Sendi, and B. Khalaji Emamzadeh Abbasi (2022), Anomaly Detection on OpenStack Logs Based on an Improved Robust PCA Model and Its Projection onto Column Space, ...,